```
// ==================================================================
// ROTINA SEMANAL - FEEDBACK HUMANIZADO E DETALHADO (MULTILOJA)
// ==================================================================

const SETTINGS_SEMANAL = {
  ABA_DADOS: 'BASE_OFICIAL',
  ABA_CONFIG: 'CONFIG',
  // Metas de Conversão (Benchmarks)
  BENCH_LEAD_AGEND: 0.20, // 20%
  BENCH_AGEND_VISIT: 0.60, // 60% (3 de 5)
  BENCH_VISIT_VENDA: 0.33  // 33% (1 de 3)
};

function gerarRelatorioSemanalMultiloja() {
  const ss = SpreadsheetApp.getActiveSpreadsheet();
  const sheetDados = ss.getSheetByName(SETTINGS_SEMANAL.ABA_DADOS);
  const sheetConfig = ss.getSheetByName(SETTINGS_SEMANAL.ABA_CONFIG);

  if (!sheetDados || !sheetConfig) {
    Logger.log("ERRO CRÍTICO: Abas BASE_OFICIAL ou CONFIG não encontradas.");
    return;
  }

  // 1. PERÍODO (Semana Anterior Fechada)
  const hoje = new Date();
  const dataFim = new Date(hoje);
  dataFim.setDate(hoje.getDate() - 1); // Ontem
  const dataInicio = new Date(dataFim);
  dataInicio.setDate(dataFim.getDate() - 6);

  Logger.log(`Processando período: ${formatDate(dataInicio)} a
${formatDate(dataFim)}`);

  // 2. CONFIGURAÇÕES
  const dadosConfig = sheetConfig.getDataRange().getValues();
  const lojasConfig = {};

  for (let c = 1; c < dadosConfig.length; c++) {
    const nomeLoja = String(dadosConfig[c][0]).trim().toUpperCase();
    const emailGerente = dadosConfig[c][1];
    const meta = Number(dadosConfig[c][2]) || 0;
```

```
    if (nomeLoja && emailGerente) {
      lojasConfig[nomeLoja] = { email: emailGerente, meta: meta };
    }
  }

  // 3. PROCESSAMENTO DE DADOS
  const dadosBase = sheetDados.getDataRange().getValues();
  const dadosProcessados = {};

  for (let i = 1; i < dadosBase.length; i++) {
    const row = dadosBase[i];
    if (!row[0]) continue;

    const rowDate = new Date(row[0]);
    rowDate.setHours(0,0,0,0);
    const dIni = new Date(dataInicio); dIni.setHours(0,0,0,0);
    const dFim = new Date(dataFim); dFim.setHours(0,0,0,0);

    if (rowDate >= dIni && rowDate <= dFim) {
      const loja = String(row[1]).trim().toUpperCase();
      if (!lojasConfig[loja]) continue;

      if (!dadosProcessados[loja]) dadosProcessados[loja] = {};
      const vendedor = String(row[2]).trim().toUpperCase();

      if (!dadosProcessados[loja][vendedor]) {
        dadosProcessados[loja][vendedor] = { leads: 0, agend: 0, visitas: 0, vendas: 0 };
      }

      dadosProcessados[loja][vendedor].leads += Number(row[3]) || 0;
      dadosProcessados[loja][vendedor].vendas += (Number(row[4])||0) +
(Number(row[6])||0) + (Number(row[8])||0);
      dadosProcessados[loja][vendedor].agend += (Number(row[5])||0) +
(Number(row[7])||0);
      dadosProcessados[loja][vendedor].visitas += Number(row[9]) || 0;
    }
  }

  // 4. GERAÇÃO DOS RELATÓRIOS
  const lojasParaProcessar = Object.keys(dadosProcessados);
```

```
  lojasParaProcessar.forEach(nomeLoja => {
    const vendedoresLoja = dadosProcessados[nomeLoja];
    const configLoja = lojasConfig[nomeLoja];
    const listaVendedores = Object.keys(vendedoresLoja);

    if (listaVendedores.length === 0) return;

    // A. Calcular Médias da Loja
    let somaLeads=0, somaAgend=0, somaVisitas=0, somaVendas=0;
    listaVendedores.forEach(v => {
        somaLeads += vendedoresLoja[v].leads;
        somaAgend += vendedoresLoja[v].agend;
        somaVisitas += vendedoresLoja[v].visitas;
        somaVendas += vendedoresLoja[v].vendas;
    });

    const mediaLoja = {
        leads: somaLeads / listaVendedores.length,
        agend: somaAgend / listaVendedores.length,
        visitas: somaVisitas / listaVendedores.length,
        vendas: somaVendas / listaVendedores.length
    };

    // Meta Semanal (Aprox)
    const metaSemanal = configLoja.meta > 0 ? (configLoja.meta /
listaVendedores.length / 4.3) : 0;

    // B. Criar Planilha
    const nomeArquivo = `Feedback Semanal - ${nomeLoja} - ${formatDate(dataInicio)} a
${formatDate(dataFim)}`;
    const novaSS = SpreadsheetApp.create(nomeArquivo);
    let corpoEmailTextos = "";

    listaVendedores.forEach(vend => {
      const r = vendedoresLoja[vend];

      // --- LÓGICA DE DIAGNÓSTICO ENRIQUECIDO ---
      let gargalo = "Parabéns! Seu fluxo de atendimento está excelente.";
      let acao = "Continue mantendo esses números para puxar a meta da loja.";

      const txLeadAgend = r.leads > 0 ? r.agend / r.leads : 0;
```

```
    const txAgendVisit = r.agend > 0 ? r.visitas / r.agend : 0;
    const txVisitVenda = r.visitas > 0 ? r.vendas / r.visitas : 0;

    // Cálculos de "Deficit" (O que faltou para o ideal)
    const idealAgend = Math.round(r.leads *
SETTINGS_SEMANAL.BENCH_LEAD_AGEND);
    const deficitAgend = idealAgend - r.agend;

    const idealVisitas = Math.round(r.agend *
SETTINGS_SEMANAL.BENCH_AGEND_VISIT);
    const deficitVisitas = idealVisitas - r.visitas;

    const idealVendas = Math.round(r.visitas *
SETTINGS_SEMANAL.BENCH_VISIT_VENDA);
    const deficitVendas = idealVendas - r.vendas;

    if (r.leads === 0) {
      gargalo = "Alerta Crítico: Você não recebeu Leads nesta semana.";
      acao = "Procure a gerência imediatamente para verificar a distribuição de leads.";
    }
    else if (txLeadAgend < SETTINGS_SEMANAL.BENCH_LEAD_AGEND) {
      gargalo = `Entrada do Funil: Você recebeu ${r.leads} leads e agendou ${r.agend}.
Pela métrica de 20%, deveríamos ter agendado pelo menos ${idealAgend}. Perdemos
${deficitAgend} oportunidades de contato aqui.`;
      acao = `Foco na velocidade: Nesta semana, ligue para o cliente nos primeiros 5
minutos. Revise seus áudios de apresentação para tentar recuperar esses
${deficitAgend} agendamentos.`;
    }
    else if (txAgendVisit < SETTINGS_SEMANAL.BENCH_AGEND_VISIT) {
      gargalo = `Quebra de Compromisso (Bolo): De ${r.agend} agendamentos, apenas
${r.visitas} vieram. A regra é que a cada 5, 3 venham. Deixamos de receber cerca de
${deficitVisitas} clientes na loja.`;
      acao = `Blindagem de Agenda: Ao agendar, envie a localização imediatamente. No
dia da visita, mande mensagem 2 horas antes confirmando. Precisamos garantir que
esses ${deficitVisitas} clientes apareçam.`;
    }
    else if (txVisitVenda < SETTINGS_SEMANAL.BENCH_VISIT_VENDA) {
      gargalo = `Fechamento: Recebemos ${r.visitas} visitas e fechamos ${r.vendas}.
Pela média (1 venda a cada 3 visitas), poderíamos ter fechado mais ${deficitVendas}
carros.`;
```

```
        acao = `Técnica de Mesa: O cliente veio, então ele queria comprar. Peça ajuda ao gerente na hora do fechamento para não deixar esses ${deficitVendas} clientes saírem sem comprar.`;
      }

      // Cria Aba
      const aba = novaSS.insertSheet(vend);
      criarPainelHumanizado(aba, vend, r, mediaLoja, metaSemanal, gargalo, acao);

      // Texto Zap (CORREÇÃO AQUI: Passando dataInicio e dataFim corretamente)
      const textoZap = gerarTextoZapHumanizado(vend, r, mediaLoja, metaSemanal, gargalo, acao, dataInicio, dataFim);

      corpoEmailTextos += `<div style="margin-bottom: 20px; padding: 15px; background: #f9f9f9; border-left: 4px solid #134f5c;">
        <strong>${vend}</strong><br>
        <pre style="white-space: pre-wrap; font-family: sans-serif; font-size: 12px; color: #333;">${textoZap}</pre>
      </div>`;
    });

    try { novaSS.deleteSheet(novaSS.getSheets()[0]); } catch(e){}

    // C. Enviar Email
    const mensagemHTML = `
      <div style="font-family: Arial, sans-serif;">
        <h2 style="color: #0c343d;">📊 Feedback Semanal: ${nomeLoja}</h2>
        <p><strong>Período:</strong> ${formatDate(dataInicio)} a ${formatDate(dataFim)}</p>
        <div style="text-align: center; margin: 20px 0;">
          <a href="${novaSS.getUrl()}" style="background-color: #25D366; color: white; padding: 12px 24px; text-decoration: none; border-radius: 5px; font-weight: bold;">
            📂 ABRIR RELATÓRIO COMPLETO
          </a>
        </div>
        <h3>📝 Sugestões de Mensagem (Copiar e Colar)</h3>
        ${corpoEmailTextos}
      </div>
    `;

    if (configLoja.email) {
```

```
      MailApp.sendEmail({
        to: configLoja.email,
        subject: `📊 Feedback Semanal - ${nomeLoja}`,
        htmlBody: mensagemHTML
      });
      Logger.log(`Relatório enviado para ${configLoja.email}`);
    }
  });
}

// --- FUNÇÃO VISUAL: PAINEL COM EXPLICAÇÃO DAS BOAS PRÁTICAS ---
function criarPainelHumanizado(sheet, nome, dados, medias, metaSemanal, gargalo,
acao) {
  // Cabeçalho
  sheet.getRange("A1:E1").merge().setValue(`RESUMO DO VENDEDOR: ${nome}`)
      .setBackground("#134f5c").setFontColor("white").setFontWeight("bold").setHorizo
ntalAlignment("center").setFontSize(12);

  // 1. Números Reais
  const headers = [["Leads Recebidos", "Agendamentos Feitos", "Visitas Realizadas",
"Vendas Fechadas", "Sua Meta Semanal"]];
  const values = [[dados.leads, dados.agend, dados.visitas, dados.vendas,
metaSemanal.toFixed(0)]];

  sheet.getRange("A3:E3").setValues(headers).setBackground("#d9d9d9").setFontWeig
ht("bold").setHorizontalAlignment("center");
  sheet.getRange("A4:E4").setValues(values).setHorizontalAlignment("center").setFontS
ize(11);

  // 2. Análise do Funil (O "Deveria Ter Feito")
  sheet.getRange("A6:E6").merge().setValue("ANÁLISE DE APROVEITAMENTO (REAL vs
IDEAL)").setBackground("#f1c232").setFontWeight("bold");

  const idealAgend = Math.round(dados.leads *
SETTINGS_SEMANAL.BENCH_LEAD_AGEND);
  const idealVisitas = Math.round(dados.agend *
SETTINGS_SEMANAL.BENCH_AGEND_VISIT);
  const idealVendas = Math.round(dados.visitas *
SETTINGS_SEMANAL.BENCH_VISIT_VENDA);

  const tableFunil = [
```

```
  ["Etapa do Processo", "Seu Resultado", "O Ideal Seria", "Status"],
  ["De Leads para Agendamentos", dados.agend, idealAgend, dados.agend >= idealAgend ? "✅ Bom" : "🔻 Abaixo"],
  ["De Agendamentos para Visitas", dados.visitas, idealVisitas, dados.visitas >= idealVisitas ? "✅ Bom" : "🔻 Abaixo"],
  ["De Visitas para Vendas", dados.vendas, idealVendas, dados.vendas >= idealVendas ? "✅ Bom" : "🔻 Abaixo"]
 ];

 sheet.getRange("A7:D10").setValues(tableFunil);
 sheet.getRange("A7:D7").setFontWeight("bold").setBackground("#fff2cc");
 sheet.getRange("A7:D10").setHorizontalAlignment("center").setBorder(true,true,true,true,true,true);

 // 3. Comparativo com a Equipe
 sheet.getRange("A12:E12").merge().setValue("SEU DESEMPENHO COMPARADO À MÉDIA DA EQUIPE").setBackground("#cfe2f3").setFontWeight("bold");

 const getComparacao = (meu, media) => {
  if (meu > media) return "MAIOR que a média da loja (+)";
  if (meu < media) return "MENOR que a média da loja (-)";
  return "IGUAL à média da loja (=)";
 };

 const tableComp = [
  ["Critério", "Sua Produção", "Média da Equipe", "Conclusão"],
  ["Volume de Vendas", dados.vendas, medias.vendas.toFixed(1), getComparacao(dados.vendas, medias.vendas)],
  ["Volume de Agendamentos", dados.agend, medias.agend.toFixed(1), getComparacao(dados.agend, medias.agend)],
  ["Volume de Visitas", dados.visitas, medias.visitas.toFixed(1), getComparacao(dados.visitas, medias.visitas)]
 ];

 sheet.getRange("A13:D16").setValues(tableComp);
 sheet.getRange("A13:D13").setFontWeight("bold").setBackground("#efefef");
 sheet.getRange("A13:D16").setHorizontalAlignment("center").setBorder(true,true,true,true,true,true);

 // 4. Diagnóstico em Texto (Com quebra de linha automática)
```

```
  sheet.getRange("A18").setValue("DIAGNÓSTICO DA
SEMANA:").setFontWeight("bold");
  sheet.getRange("B18:E18").merge().setValue(gargalo).setFontColor("red").setFontWei
ght("bold").setWrap(true);

  sheet.getRange("A19").setValue("ORIENTAÇÃO DE AÇÃO:").setFontWeight("bold");
  sheet.getRange("B19:E19").merge().setValue(acao).setFontStyle("italic").setWrap(true
);

  // 5. SEÇÃO: LEGENDA DAS BOAS PRÁTICAS
  const linhaLegenda = 21;
  sheet.getRange(linhaLegenda, 1, 1, 5).merge().setValue("📚 ENTENDA A CONTA
(BOAS PRÁTICAS DO
SETOR)").setBackground("#e6e6e6").setFontWeight("bold").setFontSize(10);

  const textoLegenda = "Consideramos como ideal: 20% do volume total de leads vira
agendamento | Para cada 5 agendamentos, 3 viram visitas (60%) | Para cada 3 visitas,
1 vira venda (33%).";
  sheet.getRange(linhaLegenda + 1, 1, 1,
5).merge().setValue(textoLegenda).setFontSize(9).setWrap(true).setHorizontalAlignme
nt("center").setBorder(true,true,true,true,true,true);

  // Formatação Final
  sheet.autoResizeColumns(1, 5);
  sheet.setColumnWidth(2, 150);
}

// --- FUNÇÃO TEXTO: WHATSAPP RICO ---
function gerarTextoZapHumanizado(nome, dados, medias, meta, gargalo, acao, dInicio,
dFim) {
  const idealAgend = Math.round(dados.leads *
SETTINGS_SEMANAL.BENCH_LEAD_AGEND);
  const idealVisitas = Math.round(dados.agend *
SETTINGS_SEMANAL.BENCH_AGEND_VISIT);

  const cpVendas = dados.vendas >= medias.vendas ? "📈 Acima da média" : "📉
Abaixo da média";
  const cpVisitas = dados.visitas >= medias.visitas ? "📈 Acima da média" : "📉 Abaixo
da média";

  return `👋 Olá, ${nome}. Segue seu Feedback Semanal.
```

```
📅 *Período:* ${formatDate(dInicio)} a ${formatDate(dFim)}

🔢 *SEUS NÚMEROS DA SEMANA*
• Vendas Fechadas: ${dados.vendas} (Meta Ref: ${meta.toFixed(0)})
• Leads Recebidos: ${dados.leads}
• Agendamentos: ${dados.agend}
• Visitas na Loja: ${dados.visitas}

💡 *ANÁLISE DE OPORTUNIDADE (O que deveria ter acontecido)*
• Com ${dados.leads} Leads, a boa prática diz que você faria *${idealAgend} agendamentos*.
• Com ${dados.agend} Agendamentos, a boa prática diz que você receberia *${idealVisitas} visitas*.

🏆 *COMPARATIVO COM A EQUIPE*
• Volume de Vendas: ${cpVendas} da loja.
• Volume de Visitas: ${cpVisitas} da loja.

🚨 *DIAGNÓSTICO E AÇÃO*
${gargalo}
👉 *O que fazer:* ${acao}

-------------------------
📚 *Entenda o critério:* Baseado em 20% de conversão de Leads para Agendamento; 3 visitas a cada 5 agendamentos; e 1 venda a cada 3 visitas.`;
}

function formatDate(date) {
  return Utilities.formatDate(date, Session.getScriptTimeZone(), "dd/MM");
}
// ==================================================================
// GATILHO INDEPENDENTE (Coloque no final do arquivo novo)
// Execute esta função (LIGAR_DISPARO_SEMANAL) apenas uma vez para ativar.
// ==================================================================
function LIGAR_DISPARO_SEMANAL() {
  // 1. Remove apenas gatilhos antigos deste script específico (limpeza)
  var triggers = ScriptApp.getProjectTriggers();
  for (var i = 0; i < triggers.length; i++) {
    if (triggers[i].getHandlerFunction() === 'gerarRelatorioSemanalMultiloja') {
```

```
    ScriptApp.deleteTrigger(triggers[i]);
  }
}

// 2. Cria o acionador: Toda Segunda-feira, por volta das 12:30
ScriptApp.newTrigger('gerarRelatorioSemanalMultiloja')
    .timeBased()
    .everyWeeks(1)
    .onWeekDay(ScriptApp.WeekDay.MONDAY)
    .atHour(12)
    .nearMinute(30)
    .create();

Logger.log("✅ Agendamento realizado com sucesso! O script rodará independente
toda Segunda às 12:30.");
}
```